



What you've

accomplished



Module 1: Advanced instruction writing

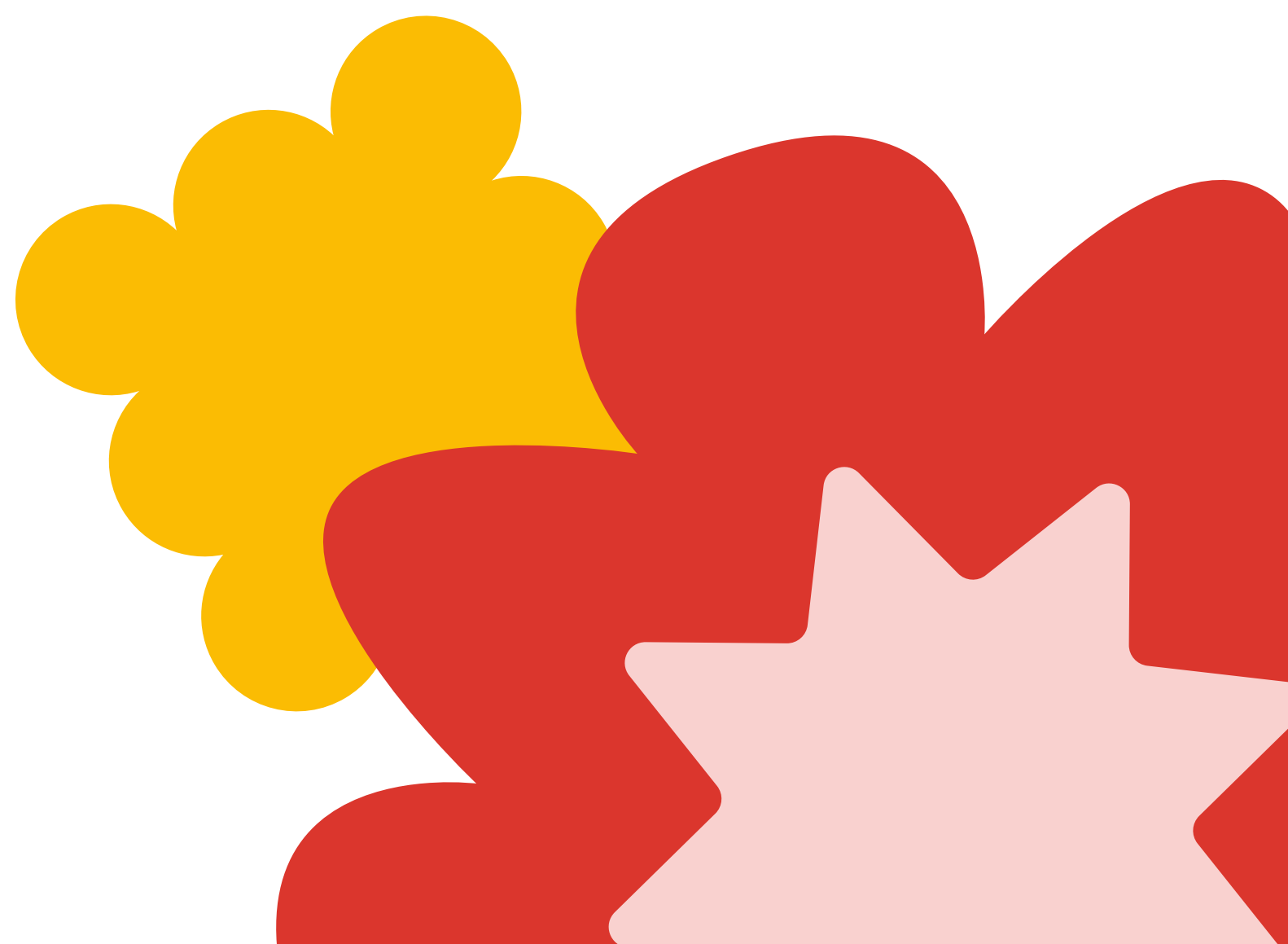
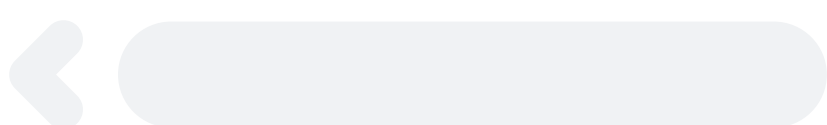
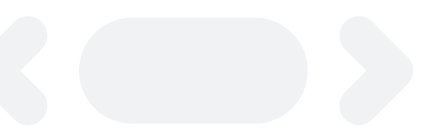
- Learned 5 reusable instruction patterns (identity, mission, methodology, boundaries, few-shot examples)
- Created domain-neutral patterns with templates and [placeholders]
- Implemented multi-domain examples across E-commerce, education, finance, healthcare
- Structured instructions with markdown organization for clarity

Key insight: Instructions are comprehensive behavior specifications. Use reusable patterns with templates to create professional agents across any domain.

Module 2: Structured output

- Defined Pydantic BaseModel schemas for predictable JSON output
- Used `output_key` for passing data in workflows
- Understood schema completeness—only defined fields appear in output
- Learned structured output works for both root and intermediate sub-agents

Key insight: Structured output bridges the gap between natural language AI and system integration. The schema is a contract—define exactly what you need.



Module 3: Choosing and configuring models

- Selected models strategically—start with Pro for prototyping, optimize with Flash
- Configured safety settings using `GenerateContentConfig` and `SafetySetting`
- Used temperature ranges for different tasks (0.0–0.3 factual, 0.8–1.0 creative)
- Learned Vertex AI pricing and model selection patterns

Key insight: Start with Gemini Pro for quality baselines and prototyping. Use Flash as an optimization for cost and speed after performing gap analysis.

Module 4: Planning for complex tasks

- Enabled multi-step reasoning with `BuiltInPlanner` and `ThinkingConfig`
- Understood planning vs. multi-agent approaches
- Compared planning output with concrete examples (buy vs. lease car)
- Learned when to use `BuiltInPlanner` (Gemini) vs. `PlanReActPlanner` (non-Gemini models)

Key insight: Planning transforms reactive agents into strategic thinkers. Use `BuiltInPlanner` for Gemini models with thinking capabilities. Simple instructions work—planning adds the structured reasoning automatically.

Real-world applications

With these skills, you can now build:

Customer service systems

```
Python
from pydantic import BaseModel, Field

class TicketOutput(BaseModel):
    ticket: dict = Field(description="Ticket details")
    priority: str = Field(description="Priority level")

LlmAgent(
    model="gemini-2.5-flash",
    instruction="[Detailed persona with boundaries]",
    planner=BuiltInPlanner(
        thinking_config=types.ThinkingConfig(include_thoughts=True,
        thinking_budget=1024)
    ),
    output_schema=TicketOutput
)
```


Data processing pipelines

Python

```
from pydantic import BaseModel, Field
from typing import List

class AnalysisOutput(BaseModel):
    insights: List[str] = Field(description="Key insights")
    metrics: dict = Field(description="Performance metrics")

LlmAgent(
    model="gemini-2.5-pro",
    instruction="[Analysis framework]",
    output_schema=AnalysisOutput
)
```

Workflow automation

Python

```
LlmAgent(
    model="gemini-2.5-flash",
    instruction="[Process rules]",
    output_key="approved",
    generate_content_config=types.GenerateContentConfig(
        temperature=0.2,
        safety_settings=[...]
    )
)
```

Educational assistants

Python

```
LlmAgent(
    model="gemini-2.5-pro",
    instruction="[Teaching methodology]",
    planner=BuiltInPlanner(
        thinking_config=types.ThinkingConfig(include_thoughts=False,
        thinking_budget=2048)
    ),
    generate_content_config=types.GenerateContentConfig(temperature=0.5)
)
```

Best practice checklist

Instruction writing

- ✓ Use five reusable patterns: Identity, mission, methodology, boundaries, few-shot examples
- ✓ Create generic templates with [placeholders] for domain flexibility
- ✓ Define clear persona with specific expertise
- ✓ Set explicit boundaries (Never/Always lists)
- ✓ Include 2-3 few-shot examples with input→output format
- ✓ Use markdown structure for readability

Model selection

- ✓ Start with Gemini 2.5 Pro for prototyping and quality baselines
- ✓ Optimize with Gemini 2.5 Flash after gap analysis for production
- ✓ Configure appropriate safety settings for your audience
- ✓ Set temperature based on task: 0.0-0.3 (factual), 0.4-0.7 (balanced), 0.8-1.0 (creative)

Planning

- ✓ Enable BuiltInPlanner for Gemini models with multi-step tasks
- ✓ Use PlanReActPlanner for non-Gemini models
- ✓ Configure ThinkingConfig with include_thoughts=True for debugging
- ✓ Set thinking_budget appropriately (512-2048 tokens typical)
- ✓ Apply realistic constraints in instructions
- ✓ Use lower temperature (0.2-0.3) for systematic planning

Structured output

- ✓ Define Pydantic BaseModel schemas (not dictionaries)
- ✓ Include ALL needed fields in schema (only defined fields appear)
- ✓ Use Field(description=...) to guide the LLM
- ✓ Use output_key for passing data between agents
- ✓ Handle optional fields with Optional[T] and defaults
- ✓ Validate output before use in production

Code patterns reference

Pattern 1: Professional service agent

Python

```
from google.adk.agents import LlmAgent, BuiltInPlanner
from google.genai import types
from pydantic import BaseModel, Field

class ServiceResponse(BaseModel):
    response: str = Field(description="The response to the user")
    action_required: bool = Field(description="Whether action is needed")
    metadata: dict = Field(default={}, description="Additional metadata")

professional_agent = LlmAgent(
    model="gemini-2.5-flash",
    name="service_pro",
    instruction="""
# PERSONA
You are [Name], a [Role] with [Experience].

# BOUNDARIES
Never: [List of restrictions]
Always: [List of requirements]

# EXAMPLES
[Few-shot examples]
""",
    planner=BuiltInPlanner(
        thinking_config=types.ThinkingConfig(include_thoughts=True,
thinking_budget=1024)
    ),
    output_schema=ServiceResponse,
    generate_content_config=types.GenerateContentConfig(
        temperature=0.5,
        safety_settings=[
            types.SafetySetting(
                category=types.HarmCategory.HARM_CATEGORY_DANGEROUS_CONTENT,
                threshold=types.HarmBlockThreshold.BLOCK_MEDIUM_AND_ABOVE
            )
        ]
    )
)
```

Pattern 2: Data extraction pipeline

```
Python
from pydantic import BaseModel, Field
from typing import List
from google.genai import types

class Entity(BaseModel):
    type: str =
    Field(description="Entity type")
    value: str =
    Field(description="Entity value")
    confidence: float =
    Field(description="Confidence score
0-1")

class ExtractionOutput(BaseModel):
    entities: List[Entity] =
    Field(description="Extracted
entities")
    relationships: List[str] =
    Field(description="Relationships
found")
    metadata: dict = Field(default={},
description="Extraction metadata")

extractor = LlmAgent(
    model="gemini-2.5-flash",
    name="data_extractor",
    instruction="Extract structured
data from text",
    output_schema=ExtractionOutput,
    generate_content_config=types.Gene
rateContentConfig(
    temperature=0.2 # Low for
consistency
)
)
```

Pattern 3: Decision making system

```
Python
from google.genai import types

decision_agent = LlmAgent(
    model="gemini-2.5-pro",
    name="decision_maker",
    instruction="\n\n"
    Evaluate requests against policy.
    Consider all factors and
dependencies.
    "\n\n",
    planner=BuiltInPlanner(
        thinking_config=types.ThinkingCo
nfig(include_thoughts=False,
thinking_budget=2048)
    ),
    output_key="approved", # Simple
boolean output
    generate_content_config=types.Genera
teContentConfig(
        temperature=0.1 # Very low for
consistency
    )
)
```


Common pitfalls to avoid

Instruction mistakes

- ✗ Vague personas without specific expertise or domain knowledge
- ✗ Missing boundaries leading to inappropriate or off-brand responses
- ✗ No few-shot examples causing inconsistent behavior patterns
- ✗ Over-complex instructions that confuse the model
- ✗ Domain-specific patterns instead of reusable templates

Configuration errors

- ✗ Starting with Flash instead of Pro for prototyping (wrong baseline)
- ✗ Using Pro for high-volume simple tasks after optimization (wastes money)
- ✗ Wrong safety settings for audience (too lenient or too strict)
- ✗ Temperature too high for factual tasks (causes hallucinations)
- ✗ Not performing gap analysis when switching from Pro to Flash

Planning problems

- ✗ Not using planner for complex multi-step tasks
- ✗ Using wrong planner type (BuiltInPlanner for non-Gemini models)
- ✗ Missing realistic constraints making plans impractical
- ✗ Over-planning simple single-step tasks
- ✗ Not setting thinking_budget appropriately
- ✗ High temperature with planning (reduces systematic thinking)

Output issues

- ✗ Using dictionaries instead of Pydantic BaseModel for schemas
- ✗ Not defining all needed fields (only defined fields appear in output)
- ✗ Missing Field(description=...) to guide the LLM
- ✗ No handling for optional/missing data
- ✗ Inconsistent field naming conventions
- ✗ Not validating output before use in production

Community and support

Primary references

- [ADK LLM Agent documentation](#): Advanced configuration and parameters
- [Gemini model documentation](#): Model capabilities and features
- [Vertex AI pricing](#): Cost optimization and model selection
- [Gemini safety settings](#): Safety configuration guide
- [Gemini thinking](#): Planning capabilities

Resources

From the community

Implementing Anthropic's Agent Design Patterns with Google ADK:

Learn how to apply professional agent design patterns and instruction techniques using ADK, covering persona creation and behavior crafting

Practical AI Agent Development Using ADK:

Best practices for agent development covering configuration, instructions, and deployment strategies

Building Agentic Applications with Google's ADK: A Hands-On SQL Agent Example:

A practical walkthrough of building production-ready agents with structured outputs and real-world integration

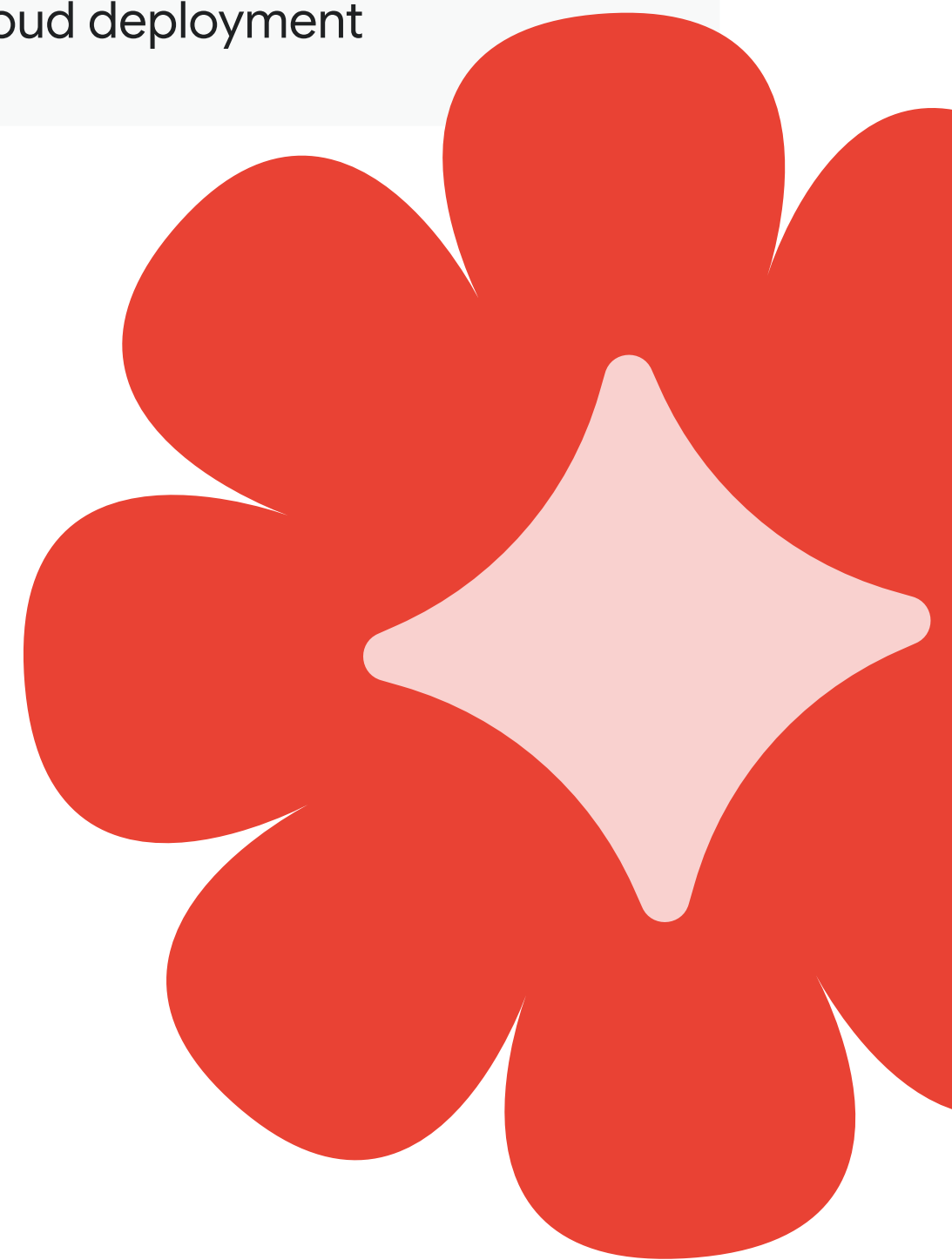
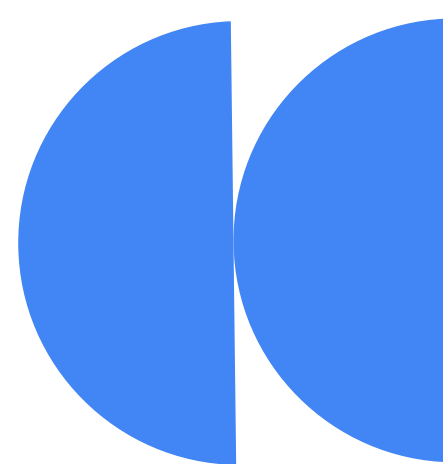
Fast-Track Your Agentic AI Journey with Google's ADK: Hands-On from Code to Cloud:

A comprehensive, hands-on guide taking you from basic agent creation through Gemini configuration to cloud deployment

Fun with agentic workflows in ADK:

Explore planning capabilities and multi-step reasoning with workflow orchestration patterns

② Questions? Post questions in the [community forum](#).



Quick reference card

Model selection strategy

None

1. Start with Gemini 2.5 Pro:
Prototyping, quality baselines,
complex reasoning

2. Optimize with Gemini 2.5 Flash:
Production, high-volume, cost
optimization

3. Always perform gap analysis when
switching Pro → Flash

When to use what

None

BuiltInPlanner: Gemini models, multi-
step tasks, dependencies

PlanReActPlanner: Non-Gemini models
with structured planning

output_schema: Pydantic BaseModel for
structured JSON output

output_key: Passing data between
agents, simple value extraction

Temperature guide

None

0.0-0.3: Facts, data extraction,
analysis, consistency

0.4-0.7: Balanced, general assistance,
customer support

0.8-1.0: Creative writing,
brainstorming, marketing

Instruction patterns

None

Identity: [Name], [Role], [Expertise]

Mission: What the agent does

Methodology: How the agent works

Boundaries: Never/Always lists

Few-Shot: Input→Output examples

Safety thresholds

None

BLOCK_LOW_AND_ABOVE: Strictest -
children, public-facing

BLOCK_MEDIUM_AND_ABOVE: Standard -
business, general use

BLOCK_ONLY_HIGH: Relaxed - research,
internal tools



Final thoughts

You've transformed from building simple agents to architecting sophisticated AI systems. Your agents now:

- Think with personality and boundaries
- Plan complex multi-step solutions
- Produce system-ready structured data
- Operate safely and efficiently

The foundation is set.

